



Finance Dashboard System

Technical Documentation

Built on Odoo 18 | PostgreSQL | OWL Framework

Module	finance_dashboard_system
Version	1.0.0
Odoo Version	18.0
Database	PostgreSQL (via Odoo ORM)
Author	Finance Dev Team
Date	3 April 2026

Table of Contents

Table of Contents 2

1. Project Overview..... 4

 1.1 Key Features 4

 1.2 Module Structure 4

2. Setup & Installation 5

 2.1 Prerequisites 5

 2.2 Installation Steps 5

 2.3 Accessing the Module 5

3. Data Model 6

 3.1 finance.record..... 6

 3.2 finance.dashboard..... 6

4. API Reference 7

 4.1 REST Endpoints..... 7

 Query Parameters 7

 Example Request 7

 Example Response..... 7

 4.2 ORM Methods (Frontend)..... 8

 Frontend Call Example 8

5. Validation & Error Handling 9

 5.1 Field-Level Constraints (@api.constrains) 9

 5.2 Soft Warnings (@api.onchange)..... 9

 5.3 Write / Delete Protection..... 9

 5.4 Error Types Used 9

6. Data Persistence..... 10

 6.1 Database..... 10

 6.2 Schema 10

 6.3 ORM to PostgreSQL Type Mapping 10

 6.4 Raw SQL (Dashboard Queries)..... **Error! Bookmark not defined.**

 6.5 Backup **Error! Bookmark not defined.**

7. Frontend (OWL Dashboard)..... 11

 7.1 Component Overview 11

 7.2 Services Used 11

 7.3 Chart Library..... 11

 7.4 Navigation 11

8. Assumptions & Tradeoffs 12

8.1 Assumptions..... 12

8.2 Tradeoffs 12

9. Known Limitations..... 13

1. Project Overview

The Finance Dashboard System is a custom Odoo 18 module that provides income and expense tracking with a real-time dashboard. It is designed to demonstrate backend service logic, REST endpoints, OWL-based frontend components, and PostgreSQL-backed data persistence.

1.1 Key Features

- Create, view, and manage financial records (income/expense)
- Real-time dashboard with summary cards, charts, and recent activity
- Category-wise totals and monthly/weekly trend graphs
- Clickable dashboard cards that open filtered list views
- REST API endpoints for dashboard data
- Input validation and error handling on all backend operations

1.2 Module Structure

```
finance_dashboard_system/  
├── __init__.py  
├── __manifest__.py  
├── models/  
│   ├── finance_record.py      # Main data model  
│   └── finance_dashboard.py    # Dashboard service (AbstractModel)  
├── controllers/  
│   └── main.py                # REST endpoints  
├── views/  
│   ├── finance_record_views.xml      # Form + List views + Search views  
│   └── finance_menu_and_action.xml    # Menus + Client action  
├── security/  
│   ├── finance_security.xml          # Record rule  
│   ├── ir.model.access.csv           # Access rights  
│   └── res_groups.xml                 # group define  
└── static/src/components/  
    ├── finance_dashboard.js          # OWL Component  
    ├── finance_dashboard.xml         # OWL Template  
    └── finance_dashboard.scss        # Styles
```

2. Setup & Installation

2.1 Prerequisites

- Odoo 18.0 installed and running
- PostgreSQL 14+ configured
- Python 3.10+

2.2 Installation Steps

Step 1 — Copy module into addons path:

```
| cp -r finance_dashboard_system /opt/odoo-custom-addons/
```

Step 2 — Update addons path in odoo.conf:

```
| addons_path = /opt/odoo/odoo/addons,/opt/odoo-custom-addons
```

Step 3 — Install the module:

```
| python odoo-bin -d your_db -i finance_dashboard_system
```

Step 4 — Upgrade (after code changes):

```
| python odoo-bin -d your_db -u finance_dashboard_system
```

2.3 Accessing the Module

- Login to Odoo backend
- Navigate to Finance menu in the top navigation bar
- Click Dashboard to view the live dashboard
- Click Records to manage financial entries

3. Data Model

3.1 finance.record

The primary model that stores all financial transactions. Maps to the `finance_record` table in PostgreSQL.

Field	Type	Required	Description
amount	Monetary	Yes	Transaction amount, linked to <code>currency_id</code>
currency_id	Many2one (res.currency)	Yes	Currency, defaults to company currency
type	Selection	Yes	income or expense
category	Char	No	Category label, 2-100 characters
date	Date	Yes	Transaction date, defaults to today
notes	Text	No	Additional notes, max 1000 characters

3.2 finance.dashboard

An `AbstractModel` — no database table. Serves as a service layer for aggregated dashboard data. All methods are decorated with `@api.model` and called via ORM from the frontend.

4. API Reference

4.1 REST Endpoints

All endpoints use HTTP GET and return JSON. Authentication is required (Odoo session cookie).

Endpoint	Method	Description	Response
/api/finance/dashboard	GET	Full dashboard data (all below combined)	{ status, data: { summary, category_totals, recent_activity, monthly_trends, weekly_trends } }
/api/finance/summary	GET	Total income, expense, and net balance	{ status, data: { total_income, total_expense, net_balance } }
/api/finance/category-totals	GET	Breakdown per category	{ status, data: [{ category, income, expense, net }] }
/api/finance/recent-activity	GET	Last N records (default 10)	{ status, data: [{ id, date, type, category, amount, currency, notes }] }
/api/finance/monthly-trends	GET	Current year month-wise totals	{ status, data: [{ month, income, expense, net }] }
/api/finance/weekly-trends	GET	Last 8 weeks totals	{ status, data: [{ week, income, expense, net }] }

Query Parameters

Endpoint	Parameter	Type	Default	Description
/api/finance/recent-activity	limit	Integer	10	Number of records to return

Example Request

```
GET /api/finance/summary
Authorization: Session Cookie (Odoo auth required)
```

Example Response

```
{
  "status": "success",
  "data": {
    "total_income": 150000.00,
    "total_expense": 75000.00,
    "net_balance": 75000.00
  }
}
```

4.2 ORM Methods (Frontend)

These methods are called directly from the OWL frontend using the orm service.

Method	Model	Returns
get_summary()	finance.dashboard	Summary dict
get_category_totals()	finance.dashboard	List of category dicts
get_recent_activity(limit)	finance.dashboard	List of activity dicts
get_monthly_trends()	finance.dashboard	List of monthly dicts
get_weekly_trends()	finance.dashboard	List of weekly dicts
get_dashboard_data()	finance.dashboard	All above combined

Frontend Call Example

```
| const summary = await this.orm.call('finance.dashboard', 'get_summary', []);
```

5. Validation & Error Handling

All validation is implemented in the `finance.record` model. Odoo raises exceptions that are automatically shown as user-friendly dialogs in the UI.

5.1 Field-Level Constraints (@api.constrains)

Field	Rule	Error Type	Message
amount	Must be > 0	ValidationError	Amount must be greater than zero
date	Cannot be in future	ValidationError	Date cannot be in the future
category	Min 2 chars, Max 100 chars	ValidationError	Category must be 2-100 characters
notes	Max 1000 chars	ValidationError	Notes cannot exceed 1000 characters

5.2 Soft Warnings (@api.onchange)

Trigger	Condition	Warning Shown
amount change	amount > 1,000,000	Large amount warning, prompts user to verify

5.3 Write / Delete Protection

Operation	Condition	Error Type	Message
write (amount)	Record is > 1 year old	UserError	Amount cannot be modified for old records

5.4 Error Types Used

Exception	When to Use	Example
ValidationError	Data is invalid or violates a business rule	Amount <= 0, future date
UserError	Operation is not allowed in current context	Editing old record

6. Data Persistence

6.1 Database

This module uses PostgreSQL via Odoo's ORM layer. No separate database configuration is required — Odoo manages all connection pooling, schema migrations, and transactions automatically.

6.2 Schema

Model	Table	Notes
finance.record	finance_record	Stores all income/expense records
finance.dashboard	(none)	AbstractModel — no table, service logic only

6.3 ORM to PostgreSQL Type Mapping

Odoo Field	PostgreSQL Type	Column
fields.Monetary	FLOAT8 / NUMERIC	amount
fields.Many2one	INTEGER (FK)	currency_id
fields.Selection	VARCHAR	type
fields.Char	VARCHAR	category
fields.Date	DATE	date
fields.Text	TEXT	notes

7. Frontend (OWL Dashboard)

7.1 Component Overview

File	Purpose
finance_dashboard.js	OWL Component class — data fetching, chart rendering, navigation
finance_dashboard.xml	OWL Template — HTML structure using t-if, t-foreach, t-ref, t-on-click
finance_dashboard.scss	Styling — cards, tables, badges, chart container

7.2 Services Used

Service	Usage
orm	Calls finance.dashboard model methods directly
actionService	Opens list/form views on card click (doAction)

7.3 Chart Library

Chart.js is loaded via CDN (cdn.jsdelivr.net) and used to render the monthly trends bar chart. The chart canvas is accessed using OWL's t-ref / useRef pattern to avoid DOM timing issues with t-if blocks.

7.4 Navigation

UI Element	Action on Click
Total Income card	Opens finance.record list filtered by type = income
Total Expense card	Opens finance.record list filtered by type = expense
Net Balance card	Opens finance.record list with no filter (all records)
Recent Activity row	Opens the specific record's form view directly

8. Assumptions & Tradeoffs

8.1 Assumptions

- Single company setup is assumed. Multi-company support is not implemented but `currency_id` defaults to `self.env.company.currency_id`, making it straightforward to extend.
- All users with access to the Finance menu have read access to all finance records. Row-level security (record rules) is not configured in this version.
- The category field is a free-text Char field, not a Many2one relation. This was intentional for simplicity — a relational category model can be added later.
- Date validation only prevents future dates. Past date limits (e.g., locked fiscal periods) are not enforced.
- The dashboard loads all data on mount. For large datasets, pagination or lazy loading would be needed.

8.2 Tradeoffs

Decision	Chosen Approach	Tradeoff
Dashboard model type	AbstractModel	No DB table — clean separation of service logic, but cannot store state
Dashboard queries	Raw SQL	Faster aggregations, but bypasses ORM security rules — acceptable for read-only summary data
Category field	Char (free text)	Simpler setup, but no autocomplete or category management UI
Chart library	Chart.js via CDN	Quick integration, but adds external dependency — can be moved to assets for offline use
REST vs ORM calls	ORM calls from frontend	Simpler auth handling, but REST endpoints also provided for external integrations
Authentication	<code>auth="user"</code> on all endpoints	Secure — requires valid Odoo session, but cannot be accessed from external tools without login

9. Known Limitations

- No multi-currency conversion — amounts from different currencies are summed as raw numbers without conversion.
- Dashboard does not auto-refresh — user must reload the page to see new data.
- No export functionality (CSV/Excel) for records or dashboard data.
- category field has no predefined values — users must type consistently for category grouping to work correctly.
- Weekly trends use Python strftime %W (Monday-based weeks). ISO week numbering differences may appear around year boundaries.